

CMBA-FL: Communication-mitigated and blockchain-assisted federated learning for traffic flow predictions[☆]

Kaiyin Zhu^a, Mingming Lu^{a,*,}, Haifeng Li^b, Neal N. Xiong^c, Wenyong He^a

^a School of Computer Science and Engineering, Central South University, Changsha, 410083, China

^b School of Geosciences and Info-Physics, Central South University, Changsha, 410083, China

^c Department of Computer Science and Mathematics, Sul Ross State University, Alpine, TX 79830, USA

ARTICLE INFO

Keywords:

Blockchain
Communication mitigating
Federated learning
Traffic flow prediction

ABSTRACT

As an effective strategy to address urban traffic congestion, traffic flow prediction has gained attention from Federated-Learning (FL) researchers due to FL's ability to preserving data privacy. However, existing methods face challenges: some are too simplistic to capture complex traffic patterns effectively, and others are overly complex, leading to excessive communication overhead between cloud and edge devices. Moreover, the problem of single point failure limits their robustness and reliability in real-world applications. To tackle these challenges, this paper proposes a new method, CMBA-FL, a Communication-Mitigated and Blockchain-Assisted Federated Learning model. First, CMBA-FL improves the client model's ability to capture temporal traffic patterns by employing the Encoder-Decoder framework for each edge device. Second, to reduce the communication overhead during federated learning, we introduce a verification method based on parameter update consistency, avoiding unnecessary parameter updates. Third, to mitigate the risk of a single point of failure, we integrate consensus mechanisms from blockchain technology. To validate the effectiveness of CMBA-FL, we assess its performance on two widely used traffic datasets. Our experimental results show that CMBA-FL reduces prediction error by 11.46%, significantly lowers communication overhead, and improves security.

1. Introduction

With the development of urbanization, both the scale of cities and population density have increased rapidly. As a result, the demand for self-driving commuting, carpool commuting, and bus commuting has also grown dramatically, which in turn has created traffic congestion and environmental pollution from vehicle emissions. Thus, Intelligent Transportation Systems (ITS) [1,2] have received increasing attention. As a vital tool of ITS, Traffic Flow Prediction (TFP) has been widely used in various applications, such as route planning [3], trip navigation [4], and traffic control and management [5] as it can improve the efficiency of traffic systems by providing timely and accurate traffic flow information.

Existing TFP works can be generally divided into two categories: traditional machine-learning-based methods and deep-learning-based methods. The latter has received increasing attention because deep learning models can automatically extract essential spatiotemporal features, which significantly improves TFP performance. However, most

of the deep-learning-based TFP works [6] only consider the centralized scheme based on the data collected through edge sensing devices. This centralized scheme has the following two problems: (1) it does not take advantage of the computing resources in the edge sensing devices; (2) it may incur data leakage due to the transmission of data from edge sensing devices to the cloud.

To address the problems of centralized TFP models, a few works [7,8] have proposed decentralized TFP models based on Federated Learning (FL) [9], which can utilize the computing power of edge sensing devices without data exchange between the cloud and edge devices, resulting in better security. However, the existing works adopt either the GRU model, which is relatively simple and has trouble characterizing the essential temporal pattern of the local data in each client [7] or introduce a GNN model and a GRU-based seq2seq model to the server and each client, respectively, leading to significant communication overhead between the clients and the server [8]. In addition, the vanilla FL [7,8] also suffers from the single point of failure problem [10]. Specifically, the vanilla FL relies on a single server (parameters aggregator),

[☆] Peer review under the responsibility of the Chongqing University of Posts and Telecommunications.

* Corresponding author.

E-mail address: mingminglu@csu.edu.cn (M. Lu).

which is vulnerable to server malfunction. Furthermore, it means any compromised server will fail all FL systems. For this reason, the question of how to relieve this failure is an urgent problem. Fortunately, Blockchain [11], which forms a decentralized distributed ledger through cryptography and peer-to-peer communication, has already been widely used in various applications, such as the Internet of Things [12], and healthcare [13]. Hence, this paper considers utilizing its decentralized properties to solve the single-point-of-failure problem of FL.

To overcome the above challenges, we propose a novel traffic flow prediction method: Communication-Mitigated and Blockchain-Assisted Federated Learning (CMBA-FL). First, we use the Encoder-Decoder structure as the model architecture of each client, which effectively improves the model's ability to represent the temporal features in the traffic flow data and reduces the overall prediction error. Second, to reduce the communication overhead during the training stage, we introduce a parameter relevance detector, which avoids the additional communication overhead caused by irrelevant client parameter uploads and also ensures competitive accuracy. Third, by leveraging blockchain, we remove the traditional single server and enable the aggregation process in FL to be completed at one random miner at each epoch. This effectively overcomes the single point of failure problem.

The main contributions of this paper are summarized as follows:

- We propose a communication-mitigated federated-learning-based traffic flow prediction method, CM-FedSeq2Seq.
- Based on CM-FedSeq2Seq, we introduce consortium blockchain technology into FL and further propose a federated learning traffic flow prediction method with superior security, CMBA-FL.
- To test the effectiveness of our methods, we conduct experiments on two public traffic flow datasets. Compared to typical-centralized methods and existing federated learning methods, CMBA-FL achieves better prediction performance, lower communication overhead, and more security.

The remainder of this article is organized as follows. In Section 2, we review the related literature on traffic speed forecasting approaches, FL, traffic speed forecasting in FL, and the Blockchain for FL. Section 3 elaborates on the design of CMFedSeq2Seq, and Section 4 discusses the design of CMBA-FL. We analyze the results in Section 5, and this article is concluded in Section 6 with a summary of potential future studies.

2. Related works

The CMBA-FL method derives elements from deep learning and FL, so we divide the related work into four parts: 1) Traditional Approaches to TFP, 2) FL approaches to TFP, 3) FL for ITS, and 4) the Blockchain for FL.

2.1. Traditional approaches to TFP

The existing research methods for TFP can generally be divided into two categories: machine learning-based methods and deep learning-based methods. Among the machine-based learning methods, Jin et al. [14] combine Principal Component Analysis (PCA) [15] and Support Vector Regression (SVR) [16] to predict traffic flows, and Tang et al. [17] use denoising algorithms to augment SVR prior to proposing a fuzzy C-mean neural network that effectively improves the prediction accuracy. However, TFP is often influenced by many factors, so it is not sufficient to rely on information such as traffic speed alone. Therefore, to research the multivariate problem TFP, Yan et al. [18] use a weighted Frobenius paradigm to estimate the similarity between multivariate time series, where the magnitude of the weights of each variable are determined by PCA. Although these methods are valuable for specific datasets, they still cannot accurately capture the complex spatio-temporal feature and dynamic correlations in traffic flow data [19].

In recent years, with the improvement of artificial intelligence, deep learning methods have achieved huge success in TFP. Among them, the two most common methods are Convolutional Neural Networks (CNN) and Recurrent Neural Networks (RNN). In terms of CNN, Zhang [20] proposed a deep learning framework based on CNN for short-term TFP that greatly improves the prediction accuracy in TFP. As for RNN, Belhadi [21] proposed to use RNN to learn potential representations of congested-roadway time series for traffic congestion prediction. Since then, many researchers have proposed a few variants, such as LSTM [22,23], GRU [24], FC-LSTM [25]. However, these methods do not consider traffic flow data's potential spatial and temporal characteristics.

To exploit the spatial-temporal characteristics of traffic flow data more fully, Lv et al. [26] use a stacked autoencoder model to learn the potential spatio-temporal characteristics of traffic flow data. Subsequently, Graph Neural Networks (GNN) [27] have shown their superior performance on various learning tasks with graph-structured data. Therefore, some researchers [28,29] have started to utilize GNN to model the natural network topology in roads. For example, Fang [28] proposes a Spatio-temporal module approach to extract spatiotemporal information from traffic flows that relies on combining multi-resolution temporal modules and globally correlated-spatial modules to encode historical traffic data. Wang et al. [29] further argues that most existing graph-based CNNs ignore the dynamic nature unique to temporal data, and then proposes a dynamic spatio-temporal graph-based CNN. The method is divided into two branches: the graph prediction branch and the TFP branch. The graph prediction branch is used to predict the structural features of dynamic graphs, and the predicted graph structural features are subsequently input to the TFP branch to complete the TFP. Guo et al. [30] propose a novel attention-based spatio temporal graph convolutional network method, that effectively improves the TFP accuracy by modeling the three temporal attributes of traffic flow data separately.

Considering the existence of serial correlation among traffic flow data, some studies [29,31] further add RNN models to encode historical data. For instance, Li et al. [29] model the traffic flow as a diffusion process on a directed graph and propose a diffusion convolutional RNN, a deep learning framework for traffic prediction that incorporates the spatial and temporal dependence of traffic flow. Wang et al. [31] firstly encode historical data using spatial GNN and then encode the whole sequence using the GRU model. Similarly, Pan [32] considers the spatial properties of road networks and uses the meta-learning method to capture the dynamic temporal dependency in traffic flow data.

2.2. FL

FL was first proposed by Google in 2016 and is a distributed method for collaborative training of edge sensing devices [33]. The goal of FL is to train a shared model in a distributed manner while ensuring that there is no data sharing between edge devices. FL can effectively avoid the risk of data privacy leakage because it does not require data sharing [34–38]. Moreover, in scenarios based on collaborative training of edge devices, since data and models are stored on edge computing nodes, decision-making relies on local rather than cloud servers, which can greatly reduce decision-making delays. Typically, there are two roles in the FL process, client and server. The edge devices used to collect data are called clients and are all able to train a local model using their collected data. The base station (or access point) is called the server, and it can aggregate and share the local model parameters trained by each edge device. The training mechanism of FL not only fully utilizes the computing power of edge devices on the edge side but also makes them more secure because they do not need to exchange data with each other. Thus, FL is now widely used in many fields. In healthcare for example [39], if ML models are trained with only the limited data available at a single hospital, some bias may be introduced into the predictions. Therefore, to make these models more general, they need to be trained

with more data, which can be achieved using FL between organizations. In finance, fraud detection is one of the most important technologies needed for banks. However, there are many limitations and challenges that hinder the development of ideal banking fraud detection systems. Due to insufficient datasets, uneven distribution, and limited detection time, existing FDSs tend to have low efficiency, low accuracy, and many false positives. Therefore, Long [40] proposes a decentralized data machine learning algorithm, the federated fraud detection framework, to train a fraud detection model with features of fraud and legitimate behavior.

2.3. FL for ITS

In ITS research, the current works rely on training raw data, which can include a lot of private information. Therefore, as a machine learning method that only shares model updates without exchanging the original data, FL is used to train algorithms on private data. Albaser [41] proposes a semi-supervised federated edge learning method to improve the recognition accuracy of the traffic sign task by introducing unlabeled data to help further train the model. As the first work [7] that applies FL to the TFP problem, FedGRU adopts the FL scheme, where the server simply aggregates the uploaded model parameters by the clients and forwards the aggregated parameters to them, which they use to update their local models. Since GRU is too simple to characterize the essential temporal pattern of traffic flows, Meng et al. [8] propose a Cross-Node Federated Graph Neural Network (CNFGNN) to replace the GRU model in each client with a GRU-based seq2seq model. Furthermore, the server in CNFGNN is attached to a GNN model, which characterizes the interaction among the client representations learned from their local models based on a predefined client-relation graph. However, the introduction of the GNN model on the server side incurs a significant increase in communication overhead between the clients and the server, which poses a great challenge to resource-constrained (such as communication bandwidth, etc.) IoT networks.

2.4. Blockchain for FL

FL in the above works still retains following security threats: (1) the centralized mechanism is vulnerable to the single-point-of-failure problem, and (2) malicious clients can upload false data about their local training updates to the parameter aggregator. As a kind of security technology, blockchain has the capability to tolerate a single point of failure by relying on a distributed consensus, and it can further implement incentive mechanisms to encourage participants to contribute to the FL network productively [42].

There has already been some work that looks at combining blockchain and FL. For example, FLChain [43] uses blockchain to achieve an unchangeable local model parameter update scheme and design an FL architecture based on a blockchain point-to-point network. Kim [44] proposes an FL method that is applied to the device based on the blockchain framework, stores the local gradient of each iteration in the block after verification and consensus, and analyzes the end-to-end delay and the optimal block generation rate. Additionally, Awan et al. [45] propose a blockchain-based privacy protection framework that eliminates the semi-honest assumptions of participants and employs encryption to protect data privacy, but it still carries the risk of a single-point-of-failure of parameter servers. Additionally, Bazzi et al. proposed a decentralized data processing approach [46], which can further enhance the trustworthiness and fault tolerance of blockchain.

3. The proposed CM-FedSeq2Seq

In TFP tasks based on FL, edge sensing devices (e.g., sensors and cameras) that collect traffic flow data are referred to as clients. Each client computes traffic data at specified time intervals and trains a local client model. Subsequently, these clients submit their local model updates to

Table 1

The summary of important symbols.

Notations	Description
r	The iteration rounds of current communication
C	The set of all clients $C = \{c_1, c_2, \dots, c_k\}$
c_k	Client $c_k, c_k \in C$
D_k	Local dataset of client c_k
w_G^r	The global model parameters on the server at the beginning of the r -th iteration
$w_{c_k}^r$	The model parameters of client c_k at the beginning of the r -th iteration
$L(w_{c_k}^r)$	The loss function of client c_k
h_c	Intermediate hidden state, which output from Encoder
X	A input sequence of traffic flow data $X = \{x_1, x_2, \dots, x_n\}$
Y	The forecast output $Y = \{\hat{y}_1, \hat{y}_2, \dots, \hat{y}_n\}$
Y_{gt}	Ground truth $Y_{gt} = \{y_1, y_2, \dots, y_n\}$

the central server, where parameter aggregation is performed iteratively until the global model achieves convergence. To facilitate understanding of the subsequent algorithms, a summary of the essential symbols is provided in Table 1.

In our CM-FedSeq2seq in order to accurately capture the temporal features of traffic flow data, a GRU-based encoder-decoder is employed as the local model for each client. FL is utilized as the training paradigm to ensure collaborative training without requiring data exchange, and the FedAVG [47] algorithm is adopted as the fundamental parameter aggregation method on the central server. Furthermore, to reduce the communication overhead of FL, we introduce a relevance-detector for clients, which mitigates overhead caused by uploading irrelevant parameters generated by the clients. In the following, we outline the workflow and describe each part of CMFedSeq2seq in detail.

3.1. CM-FedSeq2Seq

Before detailing each component, we first introduce the workflow of the CM-FedSeq2Seq framework. As shown in Fig. 1, the overall process of CM-FedSeq2Seq can be described by five key steps as follows:

- Step 1 (System initialization):** On the server, we need to set some parameters in the system, including the learning rate l_r , the number of communication rounds R_s , the number of local training rounds R_c , etc. Each unit of the global model w_G^r is initialized randomly using a standard normal distribution at round $r = 0$.
- Step 2 (The broadcast of model parameters):** The server broadcasts the global model parameters w_G^r to each client ($c_k \in C$). Then, the clients update their local models and obtain the new model parameters $\{w_{c_1}^r, w_{c_2}^r, \dots, w_{c_k}^r\}$.
- Step 3 (Local training):** Each client completes the local model training based on their local dataset D_k . Specially, they do this by minimizing the local loss function $L(w_{c_k}^r)$ to obtain the local model $w_{c_k}^{r+1}$, which is formalized in Equation (1).
- Step 4 (The upload of parameters):** Each client uploads the local model $w_{c_k}^{r+1}$ to the server.
- Step 5 (Parameter aggregation):** The server aggregates the collected local parameters using FedAVG to obtain the latest global model w_G^{r+1} , which is formalized in Equation (2).

$$w_{c_k}^{r+1} = \operatorname{argmin} L_{c_k}(w_{c_k}^r), r \in \{1, 2, \dots, R_c\} \quad (1)$$

$$w_G^{r+1} = \frac{1}{\sum_{c_k \in C} |D_k|} \sum_{c_k=1}^K |D_k| w_{c_k}^{r+1} \quad (2)$$

Steps 2–5 are repeated until the model converges.

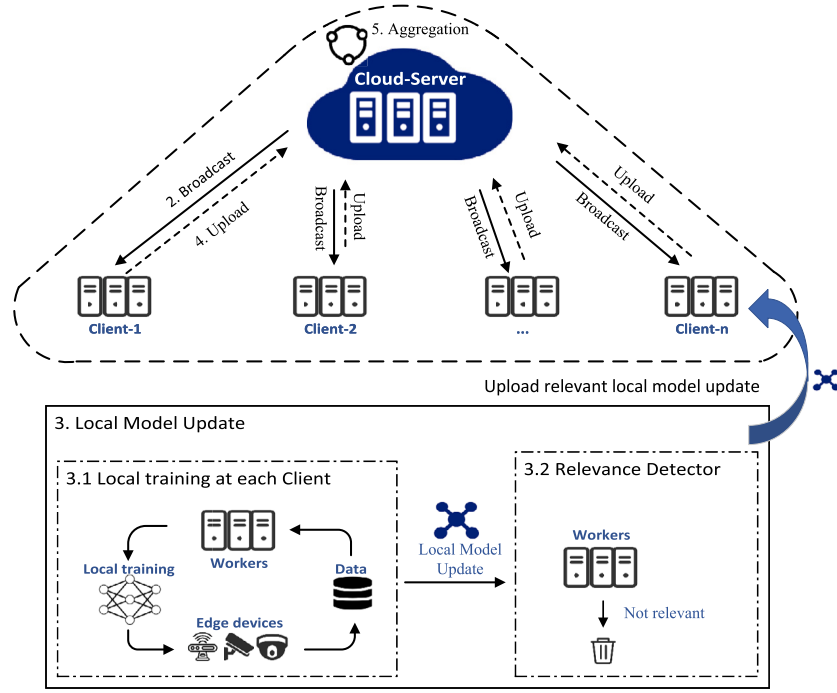


Fig. 1. The structure diagram of CM-FedSeq2Seq.

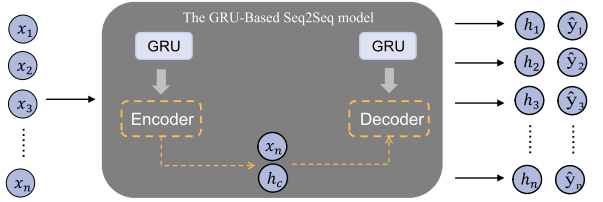


Fig. 2. The Encoder-Decoder-based client model.

3.2. Client design

The client model adopts a GRU-based encoder-decoder architecture, as illustrated in Fig. 2. This architecture is particularly well-suited for TFP, which involves complex temporal data. Traffic flow is influenced by both short-term events, such as traffic accidents or holidays, and long-term periodic patterns. Therefore, it is crucial to capture information across multiple time scales to ensure accurate predictions. The Encoder-Decoder model, with its multi-layer network structure, is capable of capturing these multi-scale features effectively. Moreover, the Encoder-Decoder architecture can accommodate different types of encoders and decoders (such as GRU, LSTM, GNN, etc.), allowing for flexibility in adapting to various data characteristics. In the federated TFP task, the model requires a strong capability to represent temporal patterns while also minimizing communication overhead. The GRU structure, though relatively simple, offers robust modeling capabilities and is well-suited for deployment on resource-constrained client devices. Consequently, we employ a GRU-based encoder to capture the temporal context information, represented by the intermediate hidden vector h_c . In the decoder module, we combine the GRU with a Fully Connected Network (FNN) to make predictions, ensuring a balance between predictive performance and computational efficiency.

3.2.1. Encoder

We use $X = \{x_1, x_2, \dots, x_n\}$, $x_i \in R^D$ as an input sequence of the traffic flow time series, where n represents the length of the series, and D denotes the feature dimensions (e.g., velocity, volume, etc.). After inputting the entire sequence X into the Encoder, it outputs an inter-

mediate hidden state vector h_c that represents the potential-dynamic temporal patterns in the input time series. In client c_k , this process can be formalized in Equation (3).

$$h_c = \text{Encoder}_k(X, h_0) \quad (3)$$

where h_0 denotes the initially hidden state vector.

3.2.2. Decoder

We denote $Y = \{\hat{y}_1, \hat{y}_2, \dots, \hat{y}_n\}$ as the output sequence of TFP, where n represents the length of the output sequence (or predicted values in n time step). In the Decoder, the input at each time step is the output of the previous time step. Note that the input of the first time step is the intermediate hidden vector h_c output from the Encoder and the last value x_n of the input sequence. In client c_k , the process is expressed in Equation (4).

$$\begin{aligned} h_1, \hat{y}_1 &= \text{Decoder}_k(x_n, h_c) \\ h_2, \hat{y}_2 &= \text{Decoder}_k(h_1, \hat{y}_1) \\ &\dots \\ h_n, \hat{y}_n &= \text{Decoder}_k(h_{n-1}, \hat{y}_{n-1}) \end{aligned} \quad (4)$$

3.3. Relevance detector

In FL, there is a heavy communication overhead incurred by the frequent parameter transmission between the clients and the server. To address this problem, we add a relevance detector before the clients upload local parameters to the server that can effectively reduce communication overhead. By calculating and comparing the local update direction of the client and the global update direction, we can determine whether it is necessary to upload the local model parameter to the server in the current round. If the difference between the local update of the client and the global update of the server is too large, we consider that the local update of the client is not helpful to the convergence of the global model. Thus, it is not necessary to upload the local model to the server. The implementation of the relevance detector is outlined as follows.

To evaluate the consistency between the local update direction of the client and the global update direction, it is necessary to compute the update directions of the client model and the global model separately. First, client c_k trains its local model based on its own local dataset, and obtains the new model parameters $w_{c_k}^{r+1}$ updated from $w_{c_k}^r$. Then, we calculate the change in each parameter in the model, formalized in Equation (5).

$$\Delta U_{c_k}^r = w_{c_k}^{r+1} - w_{c_k}^r \quad (5)$$

where $\Delta U_{c_k}^r$ denotes the model update of client c_k in the r -th round.

Similarly, we must obtain w_G^{r+1} and w_G^r to calculate the global update. However, w_G^{r+1} cannot be known in advance until all of the local clients' models $\{w_{c_1}^{r+1}, \dots, w_{c_k}^{r+1}\}$ have been aggregated in the current iteration. Therefore, it is not possible to calculate ΔU_G^r directly. However, Wang [48] experimentally found that the difference between two sequential global updates should be small. They utilized the update of the global model in the previous round as an approximate estimate of the global model in the current round, and experimentally validated this estimate. Therefore, we adopt the same strategy, which we formalize in Equation (6).

$$\Delta U_G^r \approx \Delta U_G^{r-1} = w_G^r - w_G^{r-1} \quad (6)$$

For a single parameter in a given model update, its sign determines the direction (increase or decrease) to which the model should change along the dimension of that parameter [48]. Therefore, the consistency of the two model updating directions can be measured by the sign of the parameters in the two model updates. Based on this, we measure the consistency between local updates and global updates by comparing the number of parameters with the same sign in both of them. If there are too many parameters with the opposite sign between the local update in a client and the global update, the update direction of the client is contrary to the direction of convergence of the global model. This means that the local update is not helpful and may even be detrimental to the training and convergence of the global model. If the opposite is true it means that the local update of the client is beneficial to the training and convergence of the global model. In this paper, the local update of client c_k is defined as $\Delta U_{c_k}^r = \langle u_{c_k}^1, u_{c_k}^2, \dots, u_{c_k}^M \rangle$, and the global update is defined as vector $\Delta U_G^r = \langle u_G^1, u_G^2, \dots, u_G^M \rangle$, where M represents the total number of model parameters. The computation process of consistency is formalized in Equation (7) and Equation (8).

$$e(\Delta U_{c_k}^r, \Delta U_G^r) = \frac{1}{M} \sum_{p=1}^M I(\text{sgn}(u_{c_k}^p) = \text{sgn}(u_G^p)) \quad (7)$$

$$I(\text{sgn}(u_{c_k}^p), \text{sgn}(u_G^p)) = \begin{cases} 1, & \text{if } u_{c_k}^p \text{ and } u_G^p \text{ are of} \\ & \text{the same sign} \\ 0, & \text{otherwise} \end{cases} \quad (8)$$

By setting the relevance threshold, only the local model parameters whose correlation is greater than the threshold are selected for uploading. To help the reader to understand the process of CM-FedSeq2Seq further, we detail it in Algorithm 1.

4. The proposed CMBA-FL

4.1. CMBA-FL

To address the single-point-of-failure problem inherent in FL, we propose the CMBA-FL framework, which integrates blockchain technology to ensure reliability and decentralization. Unlike vanilla FL, where parameter aggregation relies on a central aggregator, CMBA-FL leverages a blockchain-enabled community to perform secure and distributed parameter aggregation. The workflow of CMBA-FL is illustrated in Fig. 3 and is described below.

Algorithm 1 CM-FedSeq2Seq.

```

1: Input: Client set  $C = \{c_1, c_2, \dots, c_k\}$ , global iterations  $R_s$ , local training rounds  $R_c$ , learning rate  $\alpha$ , local training loss function  $\mathcal{L}(w; b)$ 
2: Output: Parameters  $\{w_{c_1}^{R_s}, \dots, w_{c_k}^{R_s}\}$ 
3: Initializing the global model  $w_G^r$ , ( $r=0$ )
4: while the global iteration round  $r = 1$  to  $R_s$  do
5:   Broadcast the parameter  $w_G^r$  to all clients in the set  $C$ .
6:   for all client  $c_k \in C$  do
7:     Initialize local model parameters  $w_{c_k}^r = w_G^r$ ;
8:      $(s_k^{r+1} 1_{c_k}, w_{c_k}^{r+1}) \leftarrow \text{ClientLocalUpdate}(R_c, w_{c_k}^r, \alpha)$ ;
9:   end for
10:   $S^r \leftarrow \{w_{c_k}^{r+1} | s_k^{r+1} \text{ is True}\}$ 
11:   $w_G^{r+1} \leftarrow \frac{1}{|S^r|} \sum_{c_k \in S^r} w_{c_k}^{r+1}$ ;
12: end while
13: Procedure  $\text{ClientLocalUpdate}(R_c, w_{c_k}^r, \alpha)$ :
14:  while local training rounds  $i = 1$  to  $R_c$  do
15:    for  $X \in \text{data}$  do ▷  $X$  represents one batch data
16:       $h_c = \text{Encoder}_k(X, h_0)$ ;
17:       $h_n, \hat{y}_n = \text{Decoder}(h_{n-1}, \hat{y}_{n-1})$ ;
18:       $w \leftarrow w - \alpha \cdot \nabla \mathcal{L}(Y, Y_{gt})$ 
19:    end for
20:  end while
21:   $s_k^{r+1} = \text{CheckRelevance}(\Delta U_{c_k}^r, \Delta U_G^r)$ 
22:  if  $s_k^{r+1}$  is False then
23:    return  $(s_k^{r+1}, \text{NULL})$  to server; ▷ irrelevant update
24:  else
25:    return  $(s_k^{r+1}, w_{c_k}^{r+1})$  to server; ▷ relevant update
26:  end if
27: Procedure  $\text{CheckRelevance}(\Delta U_{c_k}^r, \Delta U_G^r)$ :
28: ▷ equation (7) and (8)
29: Input: local update  $\Delta U_{c_k}^r$ , global update  $\Delta U_G^r$ 
30: if  $(\Delta U_{c_k}^r, \Delta U_G^r) < \text{threshold}$  then
31:   return False;
32: else
33:   return True;
34: end if

```

At the beginning of the process, a trusted task publisher initiates the training process by broadcasting the cooperative mission (e.g., TFP) to the network. This task description includes the model architecture, the required data format, and the performance goals. A group of miners is then randomly selected from the available edge devices in the network. These miners, typically implemented as edge servers, mobile base stations, or roadside units, are equipped with computational and communication resources that allow them to handle blockchain-related tasks. Their primary responsibilities include verifying client updates, participating in the consensus process, and maintaining the blockchain ledger. For their contributions, miners receive token rewards as an incentive, which also ensures their active and honest participation in the network.

Once the task is published, each client begins local training using its private dataset. The training process leverages a GRU-based Encoder-Decoder model, which is both computationally efficient and effective in handling sequential data such as traffic flow. After training, each client evaluates the relevance of its model updates using the Relevance Detector that ensures that only updates with significant contributions to model performance are submitted. The validated local updates, along with metadata such as computation time and the client's digital signature, are then uploaded to their associated miners. This step reduces unnecessary communication and prevents the server from being overwhelmed by irrelevant updates.

Upon receiving updates from the clients, each miner performs a two-step verification process to ensure the security and quality of the submitted data. First, the miner checks the digital signatures attached to the updates to confirm their authenticity. This step prevents attackers from tampering with or injecting malicious updates into the system. Second, the miner evaluates the reliability of the local model updates,

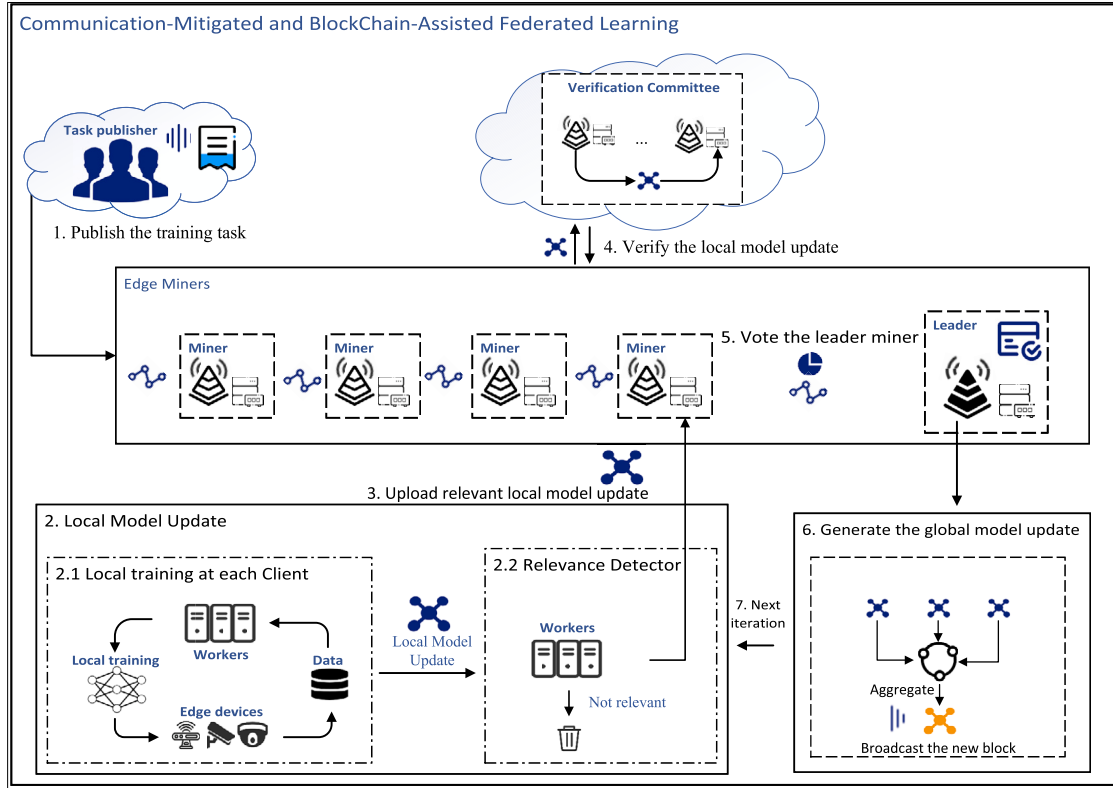


Fig. 3. The structure of CMBA-FL.

ensuring that only valid and high-quality updates are considered for aggregation. This step involves analyzing the provided metadata and possibly comparing the updates against certain pre-defined thresholds or statistical measures.

After the verification process, a leader miner is elected for the current training round using a blockchain consensus mechanism. This election process takes into account random seeds and the miners' token contributions to ensure fairness and decentralization. The elected leader is then responsible for aggregating the verified local updates using the FedAVG algorithm. This algorithm computes a weighted average of the parameters from all valid updates, producing a new global model. The use of a decentralized leader election mechanism mitigates the risks of a single point of failure, which is a significant issue in traditional FL frameworks.

Once the global model update is generated, the leader miner packages the update into a new block. This block also contains metadata such as the aggregated model's performance metrics, the list of contributing clients, and the computation logs. The block is then broadcasted across the blockchain network, where other miners validate and add it to their local copies of the blockchain ledger. Clients download the new block, extract the updated global model, and integrate it into their local systems. The iterative training process then continues, with clients using the new global model as the basis for their next round of local training.

This iterative process is repeated until the global model achieves convergence, as measured by pre-defined validation metrics. By leveraging the decentralized and tamper-proof nature of the blockchain, the CMBA-FL framework ensures secure, reliable, and efficient parameter aggregation, even in highly distributed and resource-constrained environments. The blockchain ledger also serves as an immutable record of the training process, enabling transparency and accountability for all participants.

4.2. Complexity analysis

On the client side, each client performs local training using a GRU-based Encoder-Decoder architecture. Let N denote the number of sam-

ples in the local dataset and H the number of hidden units. The computational complexity for each client is $O(\text{epochs} \cdot N \cdot H)$. Additionally, each client updates the model and uploads its parameters to the edge device, which has a communication complexity of $O(\Omega)$, where Ω represents the number of parameters. Our client model adopts a lightweight GRU-based Encoder-Decoder architecture, resulting in a small Ω , which helps reduce the overall computational complexity and communication overhead.

On the edge-side, each edge device aggregates model parameters from the clients it manages. Let $N_{clients}$ denote the number of clients managed by an edge device, the communication complexity is thus $O(N_{clients} \cdot \Omega)$. The relevance detector effectively reduces the overall communication complexity of the model by decreasing $N_{clients}$.

On the global model side, the global model aggregates the parameters received from all edge devices and sends the updated global model back. Let N_{edges} represent the number of edge devices. The communication complexity for the global model is thus $O(N_{edges} \cdot \Omega)$.

5. Performance analysis

5.1. Dataset

We performed experiments on two traffic datasets [31] to evaluate our proposed method. (1) METR-LA dataset. This traffic dataset contains traffic information collected from 207 roadway sensors on a highway in Los Angeles County. Each sensor records the speed of the vehicles every 5 minutes and contains four months of historical data from March 1, 2012, to June 30, 2012. (2) PEMS-BAY dataset. This traffic dataset is collected from 305 roadway sensors maintained by California Transportation Agencies. Each sensor records six months of data from January 1, 2017, to May 31, 2017. In this paper, we split each dataset into 70% for training, 10% for validation, and 20% for testing. Statistics of both datasets are given in Table 2.

Table 2

Classification and statistics of the two datasets.

Dataset	Number of nodes	Training dataset	Validate dataset	Test dataset
METR-LA	207	23974	3425	6850
PEMS-BAY	325	36465	5209	10419

Table 3

Prediction Performance on METE-LA Dataset.

	Method	MAE	RMSE	MAPE (%)
Centralized	ARIMA [49]	6.90	13.23	17.40
	SVR [16]	6.72	13.76	16.70
	VAR [50]	6.52	10.11	15.80
	FNN	4.49	8.69	13.20
	FC-LSTM [25]	4.37	8.60	13.20
	STGCN [51]	4.59	9.40	12.70
Other	FedGRU [7]	5.64	13.57	13.02
	CNFGNN [8]	-	11.48	-
FL	Our	4.26	10.97	9.82
	CMBA-FL	4.25	10.96	9.75

5.2. Evaluation metrics

5.2.1. Accuracy metrics

We selected three commonly-used metrics in TFP: Root Mean Square Error (RMSE), Mean Absolute Error (MAE), and Mean Absolute Percentage Error (MAPE). These metrics are formalized in Equations (9), (10) and (11), respectively.

$$RMSE = \left[\frac{1}{n} \sum_{i=1}^n (|y_i - \hat{y}_i|)^2 \right]^{\frac{1}{2}} \quad (9)$$

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (10)$$

$$MAPE = \frac{100\%}{n} \sum_{i=1}^n \left| \frac{\hat{y}_i - y_i}{y_i} \right| \quad (11)$$

where $\hat{y}_i$ denotes the predicted value, and y_i denotes the true value.

5.2.2. Communication-overhead metrics

To compare our methods with others in terms of communication overhead, we defined this overhead as the parameters delivery between all clients and the server during the training stage. We set the total number of communication rounds as R_l and the number of parameters as the size of the model. Thus, the total communication overhead can be formalized in Equation (12).

$$Comm_{cost} = 2 * R_l * nodes * size(model) \quad (12)$$

where 2 denotes the two-way communication between the server and the clients, the nodes denote the number of sensors involved in the communication, and $Comm_{cost}$ denotes the communication overhead.

5.3. Analysis of experimental results

5.3.1. Prediction accuracy

Table 3 shows the comparison results for each model on the METE-LA dataset. CM-FedSeq2Seq and CMBA-FL give the best results in several evaluation metrics compared to some typical-centralized methods. Although the RMSE of CMBA-FL is slightly worse compared to SAR, FNN, FC-LSTM, and STGCN, CMBA-FL is better each of them in both MAE and MAPE. In particular, the MAE and MAPE of CMBA-FL are 38.4% and 44.0% lower than the ARIMA, respectively. Furthermore, CM-FedSeq2Seq and CMBA-FL also outperform the two latest FL methods, FedGRU and CNFGNN. Specifically, CMBA-FL has lower error

Table 4

Prediction Performance on PEMS-BAY Dataset.

	Method	MAE	MAE RMSE	MAPE (%)
Centralized	ARIMA [49]	3.38	6.50	8.30
	SVR [16]	3.28	7.08	8.00
	VAR [50]	2.93	5.44	6.50
	FNN	4.37	8.60	13.20
	FC-LSTM [25]	2.37	4.96	5.70
	STGCN [51]	2.49	5.69	5.79
Other	FedGRU [7]	2.23	4.85	5.52
	CNFGNN [8]	-	3.82	-
FL	Our	1.73	3.75	4.05
	CMBA-FL	1.72	3.80	4.00

Table 5

The comparison of communication overhead.

Method	METR-LA		PEMS-BAY	
	RMSE	Train Comm Cost (GB)	RMSE	Train Comm Cost (GB)
FedGRU [7]	13.57	27.32	4.85	38.33
CNFGNN [8]	11.48	222.25	3.82	237.65
CM-FedSeq2Seq	10.97	52.99	3.75	68.74
CMBA-FL	10.96	36.42	3.80	48.12

metrics (MAE, RMSE, and MAPE) by 24.6%, 19.2%, and 25.1%, respectively, compared to FedGRU, and the RMSE of CMBA-FL is 4.5% lower compared to CNFGNN.

Table 4 shows the comparison results of each model on the PEMS-BAY dataset. Here, the CM-FedSeq2Seq and CMBA-FL show better results in all evaluation metrics compared to the centralized methods. In particular, CMBA-FL had lower error metrics (MAE, RMSE, MAPE) by 27.4%, 23.4%, and 29.8%, respectively, compared to FC-LSTM. In addition, CM-FedSeq2Seq and CMBA-FL both outperform the two latest FL methods, FedGRU and CNFGNN. Specifically, CMBA-FL had lower error metrics by 22.9%, 21.6%, and 27.5%, respective as above, compared to FedGRU, and the RMSE of CMBA-FL was 0.52% lower than CNFGNN.

5.3.2. Comparison of communication overhead

To highlight the advantage of the relevance detector in reducing communication overhead, we compared the communication overhead of each FL method in the training phase. As shown in Table 5, CMBA-FL ensures that the communication overhead is smaller than CNFGNN. What's more, the RMSE is lower than CNFGNN. In addition, although the communication overhead of FedGRU is slightly lower than that of CMBA-FL, its model is too simple, which leads to the prediction results of FedGRU being much worse. Therefore, the proposed method CMBA-FL does well to provide both higher accuracy and lower communication overhead compared to existing models.

5.3.3. Ablation study of client models

To evaluate the performance and communication overhead advantages of our GRU-based Encoder-Decoder client model, we also conducted an ablation study. As shown in Table 6, Methods using the Encoder-Decoder architecture perform better in terms of RMSE compared to those without it. Although more complex models such as BiGRU, BiLSTM [52], LSTM [53], and GNN generally outperform in terms of RMSE, it seems that the GRU-based Encoder-Decoder model is more efficient in terms of training communication cost, indicating a better trade-off between prediction performance and communication overhead. Thus, the GRU-based Encoder-Decoder model is more suitable for resource-constrained edge devices.

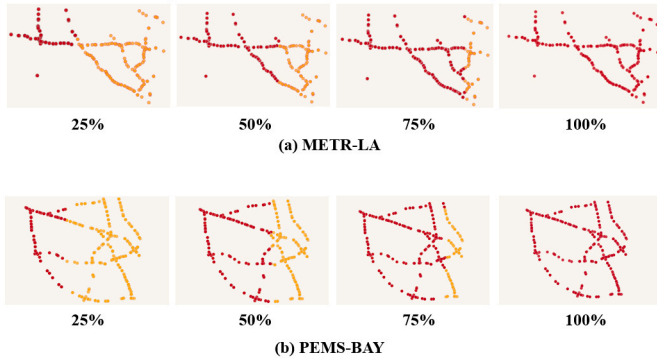
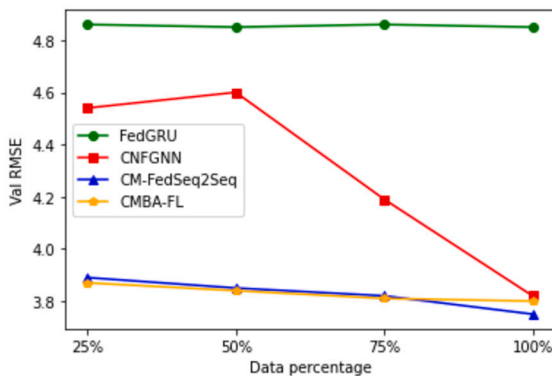
5.3.4. Generalizability at different scales

To illustrate the model fitting ability of CMBA-FL under different scales of datasets, we selected the partial sensors from METR-LA and

Table 6

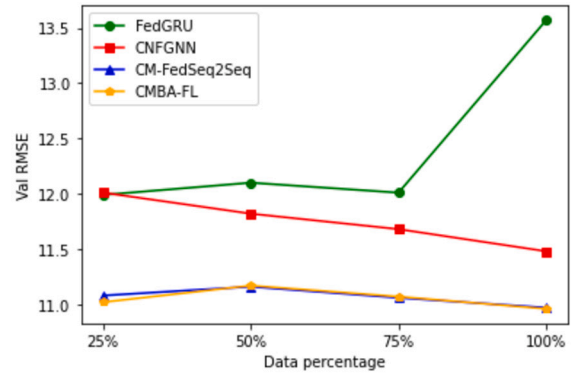
Performance and communication costs of different client models.

Client	METR-LA		PEMS-BAY	
	RMSE	Train Comm Cost (GB)	RMSE	Train Comm Cost (GB)
GRU	13.50	27.40	5.00	38.33
Encoder-Decoder (GRU)	10.97	52.99	3.75	68.74
Encoder-Decoder (BiGRU)	11.26	90.40	4.10	110.00
Encoder-Decoder (LSTM)	10.51	134.74	3.90	178.46
Encoder-Decoder (GNN)	11.48	222.25	3.82	237.65

**Fig. 4.** Visualization of the dataset. (a) shows the visualization of the METR-LA dataset in each percentage, and (b) shows the visualization of the PEMS-BAY dataset in each percentage. Among them, the red nodes indicate the distribution of selected partial nodes and the yellow nodes indicate the distribution of all nodes.**Fig. 5.** Partial Data performance is measured with Rooted Mean Squared Error (RMSE). We analyze the influence brought by different dataset scales, and compare the result of our methods with FedGRU [7] and CNFGNN [8] at METR-LA.

PEMSBAY to build a smaller dataset. We sorted all sensors based on longitudes, then used the first $\gamma\%$ of sensors as the novel-smaller datasets choosing $\gamma\% = 25\%$, $\gamma\% = 50\%$, and $\gamma\% = 75\%$ for testing. The visualization of all scale datasets is shown in Fig. 4.

As shown in Fig. 5 and Fig. 6, the RMSE of CMBA-FL and CM-FedSeq2Seq were lower than both FedGRU and CNFGNN for all different $\gamma\%$ values, thus indicating the superiority of the methods proposed in this paper. In terms of the extent to which model performance is affected by changes in dataset scale, as the γ increased, the RMSE of all CMBA-FL, CM-FedSeq2Seq, and CNFGNN all decreased, but to different extents. However, the RMSE of FedGRU even grew from $\gamma\% = 75\%$ to $\gamma\% = 100\%$ on the METR-LA dataset. This is because with an increase in data sensors, there are more and more different temporal patterns recorded on the road. The model structure of FedGRU is relatively simple, which leads to it not having enough ability to learn important temporal information. In addition, the performance of CNFGNN fluctuates more dramatically as the γ decreased, which means the dataset scale will impact its performance significantly. However, the performance of CMBA-FL and CM-FedSeq2Seq only appeared to waver slightly at smaller scales. Therefore, this testing further validates the better generalizability of our method under different dataset scales. Moreover, the traffic scale of each city is different, which means the traffic flow scales are different, which means that the deployment of our method can be carried out generically in different cities.

**Fig. 6.** Partial Data performance is measured with Rooted Mean Squared Error (RMSE). We analyze the influence brought by different dataset scales and compare the result of our methods with FedGRU [7] and CNFGNN [8] at PEMS-BAY.

tuates more dramatically as the γ decreased, which means the dataset scale will impact its performance significantly. However, the performance of CMBA-FL and CM-FedSeq2Seq only appeared to waver slightly at smaller scales. Therefore, this testing further validates the better generalizability of our method under different dataset scales. Moreover, the traffic scale of each city is different, which means the traffic flow scales are different, which means that the deployment of our method can be carried out generically in different cities.

6. Discussion

We now conduct an in-depth exploration of several critical issues in the FL environment, including consensus modeling, energy consumption, latency, security risks, robustness, and scalability.

Consensus modeling: To coordinate client updates in the global model, we introduce a correlation detector to filter out clients with abnormal update directions, thereby ensuring the overall consistency of client updates. Subsequently, we employ an averaging aggregation technique to achieve consensus among the clients. In the future, we could further optimize the consensus mechanism using attention-based weighted averaging or other methods to improve its efficiency and accuracy.

Energy consumption and latency: Our model adopts a lightweight Encoder-Decoder architecture, which reduces the communication overhead for uploading and downloading parameters. Additionally, by incorporating the parameter correlation detector, we selectively choose clients for upload, further reducing the communication overhead and computational complexity. These optimizations not only reduce energy consumption but also result in faster response times and lower latency for a given bandwidth.

Security risks: In our decentralized framework, private client data is not uploaded to a central server, ensuring its security. By integrating blockchain technology, we leverage edge servers for local model parameter aggregation while dynamically electing miners for global aggregation. This approach mitigates the risk of data loss due to the failure of a single central server as well, thereby enhancing the overall security and reliability of the system.

Robustness under different traffic conditions: The multi-layer structure of our Encoder-Decoder architecture enables our model to extract both short-term and long-term features effectively, making it well-suited to handle sudden, short-term events such as traffic congestion or accidents. Additionally, by modeling variable traffic densities, the system can adapt to fluctuations in traffic patterns. In future work, we plan to integrate data augmentation techniques to simulate diverse traffic scenarios, further enhancing the model's robustness and adaptability to complex, dynamic traffic environments.

Hardware specifications: Our experiments were conducted on a server equipped with two Intel Xeon Gold 5218 processors, 1 TB of mem-

ory, and an NVIDIA A100 GPU with 40 GB of memory. This hardware setup was used for both training and testing.

Scalability: Our model demonstrates strong scalability as the number of clients or edge devices increases. First, by adjusting the threshold of the correlation detector, we can customize client selection ratios, ensuring efficient participation in the FL process. Second, the three-tier architecture consisting of clients, edge devices, and global servers effectively distributes the pressure of parameter aggregation workload. As the number of clients grows, this hierarchical structure can be extended, with parameter aggregation occurring in multiple layers, further enhancing scalability and ensuring that the model remains efficient and effective even in large-scale deployments.

7. Conclusion

In this paper, we proposed the new CMBA-FL algorithm for TFP. The algorithm utilizes the Encoder-Decoder structure as the base model to improve temporal feature-representation ability at each edge device. In addition, we also introduced a relevance detector to prevent the upload of irrelevant parameter updates, which significantly reduces the communication overhead between edge devices and the server. Finally, we introduced consortium blockchain technology into our FL framework both to avoid the single-point-of-failure problem and to enhance data security. We conducted experiments on two common datasets and demonstrated the superior performance of our proposed method over current state-of-the-art methods.

In the future, we intend to study the following. First, we intend to consider the graph information in traffic roads in order to model temporal-spatial information. Second, we intend to introduce a layered mechanism to relieve the computational pressure on the central server and to reduce the overall communication overhead even further.

CRedit authorship contribution statement

Kaiyin Zhu: Methodology. **Mingming Lu:** Writing – review & editing, Supervision, Methodology, Funding acquisition, Conceptualization. **Wenyong He:** Writing – original draft, Software.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was partially supported by the National Natural Science Foundation of China under Grant No. U20A20182. The authors thank AiMi Academic Services (www.aimieditor.com) for English language editing and review services.

References

- [1] Y. Lin, P. Wang, M. Ma, Intelligent transportation system (ITS): concept, challenge and opportunity, in: 2017 IEEE 3rd International Conference on Big Data Security on Cloud (BigDataSecurity), IEEE International Conference on High Performance and Smart Computing (HPSC), and IEEE International Conference on Intelligent Data and Security (IDS), IEEE, 2017, pp. 167–172.
- [2] Q. Luo, Y. Zhou, Spatial-temporal structures of deep learning models for traffic flow forecasting: a survey, in: Proceedings of the Twenty-Ninth International Conference on Autonomous Systems, ICoAS, IEEE, 2021, pp. 187–193s.
- [3] K. Li, L. Chen, S. Shang, Towards alleviating traffic congestion: optimal route planning for massive-scale trips, in: Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, ACM, 2021, pp. 3400–3406.
- [4] R. Dai, S. Xu, Q. Gu, C. Ji, K. Liu, Hybrid spatio-temporal graph convolutional network: improving traffic prediction with navigation data, in: Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, ACM, 2020, pp. 3074–3082.
- [5] D. Sivabalaselvamani, Real time traffic flow prediction and intelligent traffic control from remote location for large-scale heterogeneous networking using tensorflow, *Int. J. Future Gener. Commun.* 13 (1) (2020) 1006–1012.
- [6] D. Cai, K. Chen, Z. Lin, D. Li, T. Zhou, Y. Ling, M.-F. Leung, Jointstnet: joint pre-training for spatial-temporal traffic forecasting, *IEEE Trans. Consum. Electron.* (2024), <https://doi.org/10.1109/TCE.2024.3476129>.
- [7] Y. Liu, J.J.Q. Yu, J. Kang, D. Niyato, S. Zhang, Privacy-preserving traffic flow prediction: a federated learning approach, *IEEE Internet Things J.* 7 (8) (2020) 7751–7763.
- [8] C. Meng, S. Rambhatla, Y. Liu, Cross-node federated graph neural network for spatio-temporal data modeling, in: Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, ACM, 2021, pp. 1202–1211.
- [9] L.U. Khan, W. Saad, Z. Han, E. Hossain, C.S. Hong, Federated learning for internet of things: recent advances, taxonomy, and open challenges, *IEEE Commun. Surv. Tutor.* 23 (3) (2021) 1759–1799.
- [10] H. Kim, J. Park, M. Bennis, S.-L. Kim, Blockchain-based on-device federated learning, *IEEE Commun. Lett.* 24 (6) (2020) 1279–1283.
- [11] U. Bodkhe, S. Tanwar, K. Parekh, P. Khanpara, S. Tyagi, N. Kumar, M. Alazab, Blockchain for industry 4.0: a comprehensive review, *IEEE Access* 8 (2020) 79764–79800.
- [12] D. Wang, B. Song, Y. Liu, M. Wang, Secure and reliable computation offloading in blockchain-assisted cyber-physical iot systems, *Digit. Commun. Netw.* 8 (5) (2022) 625–635.
- [13] I. Yaqoob, K. Salah, R. Jayaraman, Y. Al-Hammadi, Blockchain for healthcare data management: opportunities, challenges, and future recommendations, *Neural Comput. Appl.* 34 (14) (2022) 11475–11490.
- [14] X. Jin, Y. Zhang, D. Yao, Simultaneously prediction of network traffic flow based on pca-svr, in: Proceedings of the 4th International Symposium on Neural Networks: Part II—Advances in Neural Networks, Springer, 2007, pp. 1022–1031.
- [15] J. Lever, M. Krzywinski, N. Altman, Principal component analysis, *Nat. Methods* 14 (7) (2017) 641–642.
- [16] C.-H. Wu, J.-M. Ho, D. Lee, Travel-time prediction with support vector regression, *IEEE Trans. Intell. Transp. Syst.* 5 (4) (2004) 276–281.
- [17] J. Tang, F. Gao, F. Liu, X. Chen, A denoising scheme-based traffic flow prediction model: combination of ensemble empirical mode decomposition and fuzzy c-means neural network, *IEEE Access* 8 (2020) 11546–11559.
- [18] Y. Yan, S. Zhang, J. Tang, X. Wang, Understanding characteristics in multivariate traffic flow time series from complex network structure, *Phys. A* 477 (2017) 149–160.
- [19] A. Ali, Y. Zhu, M. Zakarya, Exploiting dynamic spatio-temporal correlations for city-wide traffic flow prediction using attention based neural networks, *Inf. Sci.* 577 (C) (2021) 852–870.
- [20] Z. Zheng, L. Shi, L. Sun, J. Du, Short-term traffic flow prediction based on sparse regression and spatio-temporal data fusion, *IEEE Access* 8 (2020) 142111–142119.
- [21] A. Belhadi, Y. Djenouri, D. Djenouri, J.C.-W. Lin, A recurrent neural network for urban long-term traffic flow forecasting, *Appl. Intell.* 50 (10) (2020) 3252–3265.
- [22] F. Althché, A. de La Fortelle, An lstm network for highway trajectory prediction, in: 2017 IEEE 20th International Conference on Intelligent Transportation Systems, ITSC, IEEE, 2017, pp. 353–359.
- [23] X. Jin, S. Liu, J. Kong, Y. Bai, T. Su, H. Ma, Gps tracking based on stacked-serial lstm network, *Chin. J. Inf. Fusion* 1 (1) (2024) 50–62.
- [24] J. Chung, C. Gulcehre, K. Cho, Y. Bengio, Empirical evaluation of gated recurrent neural networks on sequence modeling, *arXiv:1412.3555*, 2014.
- [25] J. Zhao, F. Deng, Y. Cai, J. Chen, Long short-term memory - fully connected (lstm-fc) neural network for pm2.5 concentration prediction, *Chemosphere* 220 (2019) 486–492.
- [26] Y. Lv, Y. Duan, W. Kang, Z. Li, F.-Y. Wang, Traffic flow prediction with big data: a deep learning approach, *IEEE Trans. Intell. Transp. Syst.* 16 (2) (2015) 865–873.
- [27] G. Dong, M. Tang, Z. Wang, J. Gao, S. Guo, L. Cai, R. Gutierrez, B. Campbell, L.E. Barnes, M. Boukhechba, Graph neural networks in IoT: a survey, *ACM Trans. Sens. Netw.* 19 (2) (2023) 1–50.
- [28] S. Fang, Q. Zhang, G. Meng, S. Xiang, C. Pan, Gstnet: global spatial-temporal network for traffic flow prediction, in: Proceedings of the 28th International Joint Conference on Artificial Intelligence, AAAI, 2019, pp. 2286–2293.
- [29] X. Wang, Y. Ma, Y. Wang, W. Jin, X. Wang, J. Tang, C. Jia, J. Yu, Traffic flow prediction via spatial temporal graph neural network, in: Proceedings of the Web Conference 2020, ACM, 2020, pp. 1082–1092.
- [30] S. Guo, Y. Lin, N. Feng, C. Song, H. Wan, Attention based spatial-temporal graph convolutional networks for traffic flow forecasting, in: Proceedings of the Thirty-Third AAAI Conference on Artificial Intelligence and Thirty-First Innovative Applications of Artificial Intelligence Conference and Ninth AAAI Symposium on Educational Advances in Artificial Intelligence, AAAI, 2019.
- [31] Y. Li, R. Yu, C. Shahabi, Y. Liu, Diffusion convolutional recurrent neural network: data-driven traffic forecasting, *arXiv:1707.01926*, 2018.
- [32] Z. Pan, Y. Liang, W. Wang, Y. Yu, Y. Zheng, J. Zhang, Urban traffic prediction from spatio-temporal data using deep meta learning, in: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, ACM, 2019, pp. 1720–1730.
- [33] J. Konečný, H.B. McMahan, F.X. Yu, P. Richtárik, A.T. Suresh, D. Bacon, Federated learning: strategies for improving communication efficiency, *arXiv:1610.05492*, 2017.

- [34] F. Xia, R. Hao, J. Li, N. Xiong, L.T. Yang, Y. Zhang, Adaptive gts allocation in iee 802.15.4 for real-time wireless sensor networks, *J. Syst. Archit.* 59 (10, Part D) (2013) 1231–1242.
- [35] P. Kumar, R. Kumar, G. Srivastava, G.P. Gupta, R. Tripathi, T.R. Gadekallu, N.N. Xiong, Ppsf: a privacy-preserving and secure framework using blockchain-based machine-learning for iot-driven smart cities, *IEEE Trans. Netw. Sci. Eng.* 8 (3) (2021) 2326–2341.
- [36] C. Wu, C. Luo, N. Xiong, W. Zhang, T.-H. Kim, A greedy deep learning method for medical disease analysis, *IEEE Access* 6 (2018) 20021–20030.
- [37] H. Cheng, Z. Xie, Y. Shi, N. Xiong, Multi-step data prediction in wireless sensor networks based on one-dimensional cnn and bidirectional lstm, *IEEE Access* 7 (2019) 117883–117896.
- [38] Y. Yao, N. Xiong, J.H. Park, L. Ma, J. Liu, Privacy-preserving max/min query in two-tiered wireless sensor networks, *Comput. Math. Appl.* 65 (9) (2013) 1318–1325.
- [39] I. Dayan, H.R. Roth, Zhong, Federated learning for predicting clinical outcomes in patients with covid-19, *Nat. Med.* 27 (10) (2021) 1735–1743.
- [40] G. Long, Y. Tan, J. Jiang, C. Zhang, Federated Learning for Open Banking, Springer International Publishing, 2020, pp. 240–254.
- [41] A. Albazeer, B.S. Ciftler, M. Abdallah, A. Al-Fuqaha, Exploiting unlabeled data in smart cities using federated edge learning, in: 2020 International Wireless Communications and Mobile Computing, IWCMC, IEEE, 2020, pp. 1666–1671.
- [42] C. Ma, J. Li, L. Shi, M. Ding, T. Wang, Z. Han, H.V. Poor, When federated learning meets blockchain: a new distributed learning paradigm, *IEEE Comput. Intell. Mag.* 17 (3) (2022) 26–33.
- [43] X. Bao, C. Su, Y. Xiong, W. Huang, Y. Hu, Flchain: a blockchain for auditable federated learning with trust and incentive, in: 2019 5th International Conference on Big Data Computing and Communications, BIGCOM, IEEE, 2019, pp. 151–159.
- [44] Y.J. Kim, C.S. Hong, Blockchain-based node-aware dynamic weighting methods for improving federated learning performance, in: 2019 20th Asia-Pacific Network Operations and Management Symposium, APNOMS, IEEE, 2019, pp. 1–4.
- [45] S. Awan, F. Li, B. Luo, M. Liu, Poster: a reliable and accountable privacy-preserving federated learning framework using the blockchain, in: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, ACM, 2019, pp. 2561–2563.
- [46] A. Bazzi, M. Chafii, Secure full duplex integrated sensing and communications, *IEEE Trans. Inf. Forensics Secur.* 19 (2024) 2082–2097.
- [47] H.B. McMahan, E. Moore, D. Ramage, S. Hampson, B.A. y Arcas, Communication-efficient learning of deep networks from decentralized data, *arXiv:1602.05629*, 2023.
- [48] L. Wang, W. Wang, B. Li, Cmf: mitigating communication overhead for federated learning, in: 2019 IEEE 39th International Conference on Distributed Computing Systems, ICDCS, IEEE, 2019, pp. 954–964.
- [49] K. Gilbert, An arima supply chain model, *Manag. Sci.* 51 (2) (2005) 305–310.
- [50] G.E.P. Box, G.M. Jenkins, Time Series Analysis: Forecasting and Control, 3rd edition, Prentice Hall PTR, USA, 1994.
- [51] B. Yu, H. Yin, Z. Zhu, Spatio-temporal graph convolutional networks: a deep learning framework for traffic forecasting, in: Proceedings of the 27th International Joint Conference on Artificial Intelligence, AAAI, 2018, pp. 3634–3640.
- [52] F. Wang, X. Jin, S. Yi, Li3d-bilstm: a lightweight inception-3d networks with bilstm for video action recognition, *IECE Trans. Emerg. Topics Artif. Intell.* 1 (1) (2024) 58–70.
- [53] X. Jin, T. Sun, W. Chen, H. Ma, Y. Wang, Y. Zheng, Parameter adaptive non-model-based state estimation combining attention mechanism and lstm, *IECE Trans. Intell. Syst.* 1 (1) (2024) 40–48.